

DATW Algorithm — Code Documentation

1. Overview

DATW (Distributed Allocation with Time Windows) is a decentralized algorithm for assigning tasks to multiple UAVs (Unmanned Aerial Vehicles). Unlike centralized systems where one "master" computer decides everything, in DATW, each UAV makes its own decisions locally and then talks to its neighbors to resolve conflicts.

Key features:

- **Decentralized**: No single point of failure.
- **Time Windows**: Tasks must start within a specific time range `[Start, End]` .
- **Consensus**: UAVs agree on who does what by comparing "bids" (significance scores).

The implementation is split across these files:

File	Role
<code>`shared/utils.py`</code>	Basic math helpers (distance calculation).
<code>`shared/models.py`</code>	Defines <code>`Task`</code> and the base <code>`UAV`</code> class.
<code>`algs/datw/uav.py`</code>	The core DATW logic (<code>`DATWUAV`</code>), implementing the specific scoring for each task.
<code>`algs/runners.py`</code>	The simulation loop (<code>`ConsensusRunner`</code>) that runs the phases.
<code>`shared/simulation_utils.py`</code>	Setup and metrics calculation.

2. Data Structures

2.1 Task (`shared/models.py`)

A task represents a location a UAV must visit.

- `id` : Unique number for the task.
- `location` : `(x, y, z)` coordinates.
- `duration` : How long the UAV stays at the task.
- `time_window` : `[Early_Start, Late_Start]` . The task cannot start before `Early_Start` .

2.2 UAV State Vectors (`shared/models.py`)

Each UAV maintains four key lists (vectors) that represent its view of the world. These are updated locally and shared with others.

Vector	Name	Description
Z	Assignment Map	Who owns task j ? $Z[j] = i$ means UAV i owns task j .
Q	Significance Map	What is the "bid" or score for task j ? Lower $Q[j]$ means higher significance.
T	Time Map	When does task j start? $T[j]$ is the scheduled start time.
S	Timestamp Map	When did I last hear from UAV k ? Used to ignore old information.

3. The Core Concept: Significance (Scoring)

The heart of DATW is how a UAV decides if a task is "good" for it. This is calculated in `algs/datw/uav.py`.

3.1 Cost Function: $F(\text{path})$

The "cost" of a path (a sequence of tasks) is simply the **sum of completion times**.

$\text{Cost} = \sum (\text{Start_Time}_k + \text{Duration}_k)$ for all tasks in path

We want to **minimize** this cost.

3.2 Significance: q_{ij} (Formula 12)

"How important is this task to my current path?"

When a UAV i already has a path P_i , the significance of a specific task j in that path is:

$\text{Significance} = (\text{Cost_with_j} - \text{Cost_without_j}) * (\text{Start_Time_j} - \text{Window_Start_j})$

- **Term 1** ($\text{Cost_with_j} - \text{Cost_without_j}$): The marginal cost. How much does adding this task increase my total mission time?
- **Term 2** ($\text{Start_Time_j} - \text{Window_Start_j}$): The slack. How close to the opening time strictly can I do it?
 - Small slack = Urgent! I am doing it as early as possible.
 - Large slack = I am doing it late, so maybe someone else can do it better.

Intuition: We want small values. A small value means "I can do this task efficiently (low cost increase) AND I can do it early (low slack)."

3.3 Marginal Significance: q^*_{ij} (Formula 13)

"What is the best possible score if I add this NEW task?"

When considering a **new** task j that is NOT in the path yet, the UAV tries inserting it at every possible position k in its current path.

$\text{Marginal_Sig} = \text{Min over all positions } k \text{ of: } [(\text{New_Cost} - \text{Old_Cost}) * (\text{Start_Time}_j - \text{Window_Start}_j)]$

This function finds the **best insertion point** for the task. If the task cannot be inserted anywhere without violating time windows, it returns **infinity**.

4. The Algorithm Phases (`algs/runners.py`)

The algorithm runs in a loop until everyone agrees (Consensus).

Phase 1: Task Inclusion (Bundle Building)

Goal: Each UAV greedily adds tasks to its local bundle to improve the global score.

- **Identify Candidates:** the UAV looks at every task j in the world.
- **Calculate Improvement:**
 - Calculate q^* (best cost to add j to my path).
 - Compare q^* with $Q[j]$ (the current network-best score for this task).
 - If $q^* < Q[j]$, it means "I can do this task better than whoever currently owns it!"
- **Select Best Move:**
 - The UAV picks the one task that gives the **biggest improvement** ($Q[j] - q^*$).
- **Update:**
 - Insert the task into the path.
 - $Z[j] = \text{my_id}$ (I claim it).
 - $Q[j] = q^*$ (Update the score).
 - $T[j] = \text{start_time}$ (Update the time).

This repeats until the UAV is full or can't find any more tasks to improve.

Phase 2: Conflict Resolution (Consensus)

Goal: UAVs talk to each other to resolve conflicting claims.

- **Communication:** Every UAV sends its Z , Q , T vectors to its neighbors.
- **Update Rule (`update_state` in `shared/models.py`):**

When UAV i receives data from neighbor k regarding task j :

- **Case A: We both claim it.** ($Z[j]=i$ and $\text{Neighbor_Z}[j]=k$)
 - Compare scores ($Q[j]$).

- If `Neighbor_Q[j] < My_Q[j]` : **They win.** They have a better (lower) score. I update my `Z[j]=k` and `Q[j]=Neighbor_Q[j]` .
- If scores are equal: Tie-break using UAV IDs (smaller ID wins).
- **Case B: They claim it, I don't.** (`Z[j] != i` and `Neighbor_Z[j]=k`)
 - If their score is better than what I previously knew (`Neighbor_Q[j] < My_Q[j]`), I accept their claim. Update `Z` and `Q` .
- **Case C: They say it's unassigned.** (`Neighbor_Z[j] = -1`)
 - If I thought `k` owned it (`Z[j]=k`), but `k` says "I dropped it", then I update `Z[j] = -1` .
- **Task Removal** (`task_removal`):

After updating, I might find that I lost a task I asserted in Phase 1 (e.g., someone else had a better score).

- **Remove Lost Tasks:** If `Z[j]` is no longer `my_id` , I remove task `j` from my path.
- **Check Feasibility:** Removing a task might shift times for subsequent tasks. If this causes a time window violation strictness later in the path, drop those violating tasks too.

Phase 3: Secondary Inclusion (Reallocation)

Goal: Pick up any "leftover" tasks that no one claimed.

This runs **after** Phase 1 & 2 have fully converged (everyone agrees).

- **Identify Unassigned:** Look for tasks where `Z[j] = -1` .
- **Bid with Caution:**
 - Calculate `q*` for these tasks.
 - Pick the task with the **lowest absolute `q*`** (minimal cost increase).
 - **Crucial Difference:** In Phase 1, we maximize *improvement* vs current owner. In Phase 3, there is no owner, so we just minimize *our own cost increase*.
- **Add & Repeat:** Add the best task, update vectors, and repeat until full.

5. File Walkthrough

`shared/utils.py`

Simple math.

- `calculate_distance(a, b)` : Returns distance between two 3D points.

`shared/models.py`

The foundation.

- `Task` : The data object.

- `UAV` : The parent class. It handles:
 - `task_inclusion()` : The logic for the "greedy add" loop.
 - `update_state()` : The consensus logic (comparing Z/Q/T vectors).
 - `task_removal()` : Logic to drop tasks if we lose the bid.
 - `_calculate_times_for_sequence()` : Iterates through a list of tasks to find start/end times.

`algs/datw/uav.py` (`DATWUAV`)

The "Brain" of DATW.

- Inherits from `UAV` .
- Overrides `calculate_significance` : Implements Formula 12.
- Overrides `calculate_marginal_significance` : Implements Formula 13 (try all positions).
- Adds `secondary_inclusion` : The special Phase 3 logic.

`algs/runners.py` (`ConsensusRunner`)

The "Simulator".

- `run()` method:
 - Loop `max_iterations` :
 - Run `task_inclusion` for all UAVs.
 - Run `communicate` and `update_state` loop until consensus.
 - Check for global convergence (no changes).
 - If converged, run `secondary_inclusion` (Phase 3).
 - Calculate final metrics.

`shared/simulation_utils.py`

- `setup_scenario` : Creates random tasks and UAVs.
- `evaluate_metrics` : Checks if the final solution is valid (no time overlaps, all constraints met) and sums up the total cost.

6. Diagram

graph TD

```
Start[Start Simulation] --> Phase1[Phase 1: Task Inclusion]
Phase1 -->|Each UAV builds bundle| Phase2[Phase 2: Consensus]
Phase2 -->|Exchange Z, Q, T| Conflict{Conflict?}
Conflict -- Yes --> Resolve[Apply Consensus Rules]
Resolve --> Removal[Task Removal]
Removal --> Phase2
Conflict -- No (Converged) --> Check{New Tasks Added?}
Check -- Yes --> Phase1
Check -- No (Stable) --> Phase3[Phase 3: Secondary Inclusion]
Phase3 -->|Pick up unassigned tasks| Phase3Consensus[Consensus Round]
Phase3Consensus --> End[End & Evaluate]
```